Engine Starting Lab Programmer Guide

Emmanuel Yankson Sackey | Index 01250377M

Supervisor: Dr. Peter Nyanor

28 October 2026

This guide defines the tested virtual-platform implementation for Emmanuel Yankson Sackey's selected fingerprint and authorized-voice starting concept. The native Simulink model is the numerical reference. The JavaScript engine reproduces its sampled logic and is intended for integration into the educational platform.

1 Handoff and first checks

1. Read CODEX_Yankson_Platform_Implementation.md. It includes the complete runtime engine between explicit markers and does not require access to this conversation.

2. Preserve the native SLX, which contains the original validation CSVs. Inspect the destination platform before changing its structure.

3. Extract the JavaScript block from the Markdown prompt into the destination platform and run its --self-test command.

4. Open the SLX, locate its temporary resources with getappdata(0,'YanksonBundleRoot'), and run --verify-csv against the validation subfolder. Confirm all 33 scenarios and 28 signals agree.

5. Build the platform UI around the engine API. Keep UI timing, device adapters and rendering separate from the fixed-step control model.

The supplied engine contains no physical actuator endpoint and no ESP32 firmware. Platform work must not silently add a real starter output. Physical integration remains the second project stage.

Recorded verification

Native model: 33 scenario cases and 316 assertions passed. JavaScript parity: 52,833 samples and 1,479,324 values compared, with maximum absolute discrepancy about 5.12×10^{-13}. Use the JSON reports as the detailed evidence; do not replace native traces with generated JavaScript traces.

2 Architecture and execution order

Each step is 0.01 s. The controller's persistent state is reset only by creating a new engine or by the defined reset input. The clock is derived from the sample index; animation speed must not change the integration step.

1. Interpret finger presence and per-contact match events. Generate short, long and failed-match outputs.

2. Update authorization and voice sequence tracking using the previous supervisor state.

3. Calculate the engine from the previous step's relay commands. This explicit actuation delay breaks the feedback algebraic loop.

4. Construct and validate the demonstration CAN frame when its 20 ms period is due. Retain the last good numeric RPM but invalidate stale feedback.

5. Update running qualification from accepted RPM and its validity.

6. Execute the ordered Stateflow transition rules and immediate relay gates.

7. Record the telemetry row, then retain the current state and relay command for the next step.

State	**Code**	**Meaning**
LOCKED	0	No voice permission; valid long touch may request crank.
AUTHORIZED	1	Temporary voice permission.
CRANKING	2	Accepted crank with original start time.
RUNNING	3	Starter off; run supply retained.
FAULT	4	Outputs off; deliberate reset required.

Hard stop and supply/interlock faults have priority. Reset can clear a fault only when those conditions permit it. Qualified RPM precedes timeout in the chart, while the final relay gate independently enforces the original deadline.

3 Equation implementation map

Equations	Implementation
2.1	ys_touch: release time minus contact time; one event per contact.
2.2 and 2.3	ys_access: expiring permission, fresh voice event and long-touch request.
2.4 and 2.9	ys_relay plus Stateflow: immediate gate and earliest release condition.
2.5 and 2.6	ys_plant: Euler torque balance and rad/s to RPM conversion.
2.7 and 2.8	ys_qualify: continuous valid-speed qualification.
2.10 to 2.12	ys_can: demo byte decoding, ID/length/range/age checks and bus load.
2.13 and 2.14	ys_engineering: regulated power, battery current and conductor losses.
2.15 to 2.18	ys_engineering: caster load, section inertia, stress, deflection and tipping force.
2.19	Telemetry block and yankson_columns: named sampled outputs.
2.20 to 2.22	yankson_metrics and JavaScript metrics: recognition rates and zero-failure bound.

No recognition trial count is inferred from deterministic control scenarios. Undefined rates remain NaN in MATLAB and null in JavaScript. A reported zero-failure confidence bound applies only when its declared independent trials meet the statistical assumptions.

The plant uses an ignition latch after at least 0.35 s of starter command and 100 RPM, when ignition and run supply are allowed. This scenario rule enables the torque-balance demonstration; it is not a combustion or cranking calibration for the donor engine.

4 Parameters

P index	Name	Default
1	Ts	0.01 s
2 and 3	ShortMin and LongMin	0.15 s and 1.50 s
4 and 5	AuthLife and CrankMax	10 s and 3 s
6 and 7	RunRPM and RunQual	600 RPM and 0.20 s
8 and 9	CANStale and CANPeriod	0.10 s and 0.02 s
10 and 11	CANScale and CANOffset	1 RPM/bit and 0 RPM
12 and 13	StoppedRPM and MaxRPM	50 RPM and 8000 RPM
14	J	0.25 kg m²
15 and 16	StarterTorque and CombustionTorque	8 N m and 25 N m
17 and 18	ViscousLoss and LoadTorque	0.28 N m s/rad and 3 N m
19 and 20	IgnitionDelay and IgnitionMinRPM	0.35 s and 100 RPM
21	VoiceMaxAge	0.50 s
22	DemoCANID	256 decimal, 0x100

These settings are stored inside the model workspace. The embedded yankson_parameters.m supplies defaults. The JavaScript array uses zero-based indexing, so P index 1 corresponds to array index 0.

Changing a parameter requires a recorded configuration and new test evidence. The current parity tests validate the supplied defaults. Retain the 10 ms step unless the complete timing specification and validation are revised.

5 Input contract

Pass exactly 18 finite numbers to step(input). Boolean inputs use 0 or 1. A scenario CSV has time in its first column followed by these 18 inputs.

Position	Signal and meaning
1	fingerPresent: continuous simulated contact.
2	matchEvent: 1 matched, -1 failed, 0 no new result.
3	voiceSequence: increasing event number; duplicates are ignored.
4	voiceCaptureTime: model-clock event timestamp in seconds.
5	voiceID: 1 is the configured start command.
6 and 7	emergencyStop and reset.
8 and 9	powerGood and interlockOK; both must be 1.
10 to 13	CANdrop, badID, badDLC and badRange fault injection.
14	monitorConnected; observation only.
15	ignitionAllowed scenario input.
16	extraLoadNm: additional resisting torque.
17 and 18	overrideRPM and useRPMoverride for logic boundary tests.

A microphone adapter must use a current authorization epoch. The native panel starts listening only after permission is active, and rejects the result if that permission has changed or expired. It then stamps the newly received command with current model time. Generic integrations must preserve equivalent freshness and expiry checks.

Never accept a browser checkbox, a client-side “authenticated” flag or a replayed device response as physical identity proof. Simulated identity inputs must be visibly labelled as simulation.

6 Output and API contract

createEngine(parameters) returns step, time, parameters and columns. step advances one fixed sample and returns the 28-value row. simulate accepts timestamped input rows and rejects a clock mismatch. engineering returns eleven supporting calculations; metrics computes declared trial rates.

Row positions	**Recorded outputs**
1 to 4	time_s, state, authorized, starter
5 to 8	rpm_CAN, CAN_valid, event_code, run_supply
9 to 12	rpm_plant, short_event, long_event, failed_match
13 to 16	touch_duration_s, start_request, voice_accepted, voice_reject_reason
17 to 20	running_confirmed, qualification_s, CAN_age_s, CAN_raw
21 to 24	CAN_frame_accepted, combustion, monitor_connected, crank_start_s
25 to 28	authentication_time_s, voice_sequence, voice_timestamp_s, hard_stop

State event codes: 0 initial; 1 permission granted; 2 permission ended; 3 returned to stopped state; 10 crank accepted; 11 running confirmed; 20 hard fault; 21 timeout; 30 reset. This field retains the last state event. Voice rejection reasons are pulse values: 1 no current permission, 2 stale or invalid timestamp, 3 wrong command.

A start request may be recorded while the supervisor correctly inhibits cranking because RPM is invalid or not stopped. Always inspect start_request, CAN_valid, rpm_CAN, state and starter together.

For browser integration, use a worker or an accumulator to execute 0.01 s steps. Render at a lower independent rate. On pause, freeze simulation time and show PAUSED; do not accumulate hidden crank time or silently skip numerical steps.

7 Device adapter and integration boundaries

WindowsInputBridge.ps1 supports Probe, Listen, Wave and Fingerprint. Each response includes a request ID, status, timestamps and an explicit result. The native control desk accepts only the pending request ID. Device work runs outside the simulation timer so that speech recognition cannot freeze starter supervision.

Speech uses the installed local System.Speech recognizer with Start engine, Stop engine and Cancel command grammar alternatives. Only Start engine at confidence 0.65 or higher produces a start event. Stop engine is not mapped to the emergency-stop control; use the visible emergency-stop button for that test.

The biometric adapter uses Windows Biometric Framework to verify the current Windows user against an already enrolled fingerprint. It does not enroll, export or store raw fingerprints. An unavailable reader, timeout, cancellation or failed match does not produce a valid identity event. This machine’s compatibility probe returned zero accessible sensors.

Windows verification does not expose continuous contact timing. The native demonstration explicitly applies a selected simulated short or long gesture after verification. Physical R503 integration must separately establish finger-presence behaviour and per-contact matching.

A browser implementation needs an explicitly designed host bridge or supported authenticator flow. WebAuthn success proves an authenticator ceremony, not necessarily use of a fingerprint. Label the actual method accurately. Do not copy the Windows adapter into browser JavaScript and claim direct sensor access.

The SN65HVD230 electrical bus, ECU broadcasts, relay contacts, engine combustion and stand mechanics are not physically measured by this model. Retain these boundaries in platform labels and results.

8 Maintain the self contained model

1. Keep an unchanged copy of the four delivered files before making corrections. Open the SLX in MATLAB. Its PostLoadFcn prepares embedded helpers in a process-specific system temporary folder.

2. Inspect the UserData property of the 11 Control desk block. Its files field contains resource entries with path and bytes fields. getappdata(0,'YanksonBundleRoot') locates the extracted copies, including test evidence.

3. Change control and engine logic in the native MATLAB Function blocks or Stateflow table. Update the corresponding embedded core reference and JavaScript engine to keep the three representations consistent. Editing a cached reference file alone does not change a compiled model block.

4. For UI, test or adapter changes, edit the relevant temporary helper, then replace that resource entry’s bytes in the 11 Control desk UserData and save the SLX with UserDataPersistent on. Cache-only edits are overwritten when the model opens again.

5. Run test_yankson_model and test_live_panel after opening the SLX. Check every case and inspect the live-panel screenshot. Run the extracted JavaScript self-test and compare it against the native CSV outputs.

6. Reopen the SLX with only the four deliverables present. Run it, check the controls, then close it. Confirm that no dependency is missing and only four files remain. A temporary .slx.autosave recovery file may appear while the model is open; keep MATLAB recovery protection enabled.

The model redirects CacheFolder and CodeGenFolder only for the MATLAB session and restores their previous values when the model closes. Do not replace the temporary cache path with the delivery folder. Preserve the original evidence before intentional recalibration.

Physical integration remains a later stage requiring measured parameters, confirmed ECU/CAN definitions and controlled bench commissioning.

9 Implementation references and evidence

The project's Chapter Two supplies the equation numbering and provisional design values. The delivered MATLAB function comments repeat those mappings. No paper or hardware measurement is invented to fill a missing calibration.

MathWorks documentation used for R2024b implementation: Stateflow programmatic interface; model appearance parameters for Description in Tooltip; conversion of charts to Stateflow transition tables, available since R2023b.

https://www.mathworks.com/help/stateflow/programmatic-interface.html

https://www.mathworks.com/help/simulink/slref/model-appearance-parameters.html

https://www.mathworks.com/help/stateflow/api/converttostt.html

Microsoft documentation used for local Windows interfaces: SpeechRecognitionEngine and Windows Biometric Framework WinBioVerify. These are distinct from a generic Windows Hello consent result.

https://learn.microsoft.com/en-us/dotnet/api/system.speech.recognition.speechrecognitionengine

https://learn.microsoft.com/en-us/windows/win32/api/winbio/nf-winbio-winbioverify

Implementation documentation accessed 27–28 October 2026. The SLX embeds native input/output CSVs and validation, parity, interactive-panel and speech reports. Open the model to inspect them in its temporary resources. An unavailable hardware trial remains not performed, never a pass.

10 Review and correct Codex output

Use the Markdown prompt as the implementation brief and this guide as the review checklist. The native SLX is the numerical reference. Review changes before accepting them into the virtual platform.

If Codex changes this	Required correction
Start before authentication	Restore the short-touch voice permission and fresh-command checks.
One output for ignition and starter	Restore separate starter and run-supply signals; starter release must not stop normal running.
A new timer after CAN loss	Use the original crank start time and preserve the 3 s deadline.
Animation rate drives physics	Restore a 0.01 s control step independent of display refresh.
A PIN or checkbox counts as a fingerprint	Label the actual authentication method; simulated match is not biometric proof.
Claimed hardware or recognition accuracy	Require recorded physical trials. Keep unavailable trials marked not performed.

1. Identify the first differing sample or failing assertion, including its inputs, state, outputs and expected result.

2. Ask Codex to correct that specific behaviour without changing unrelated model rules. Require the changed files and the reason for each change.

3. Repeat the failing case, then the complete native and JavaScript comparison. Accept only when the regression is resolved and other cases still pass.

4. Verify the actual platform UI separately: contact press/release, command expiry, fault recovery, device cancellation, pause behaviour and result export.